TUGAS MATA KULIAH BIG DATA

APACHE SPARK
PENGERTIAN, PERBEDAAN DENGAN MAPREDUCE, RDD, DAG, DAN ALUR KERJA

Nama : Muhammad Anwar Ismail

NIM : 24051204161

Program Studi S1 Teknik Informatika, Universitas Negeri Surabaya

A. Pengertian Apache Spark

Apache Spark adalah perangkat lunak open source untuk mengolah data berukuran sangat besar menggunakan banyak komputer sekaligus yang tergabung dalam sebuah kluster. Spark dikembangkan di AMPLab, University of California, Berkeley pada tahun 2009 dan kini dikelola oleh Apache Software Foundation (Zaharia et al., 2012). Sebagai gambaran sederhana, bayangkan tumpukan dokumen setinggi gunung yang harus dihitung isinya. Satu orang membutuhkan waktu berhari-hari untuk menyelesaikannya. Jika tumpukan itu dibagi kepada seratus orang dan setiap orang menghafal hasil hitungannya di kepala, pekerjaan selesai dalam hitungan menit. Spark bekerja dengan cara yang sama, dan "kepala" yang dimaksud adalah memori (RAM).

Prinsip utama Spark adalah menyimpan data di dalam RAM selama mungkin sehingga komputer jarang membaca dan menulis ke disk yang prosesnya lambat. Berkat prinsip ini, Spark dapat bekerja hingga 10 sampai 100 kali lebih cepat daripada Hadoop MapReduce (Program Studi S1 Teknik Informatika, 2026). Spark juga disebut sebagai unified computing engine, yaitu satu mesin yang menyatukan berbagai kebutuhan analisis data (Chambers and Zaharia, 2018). Komponen-komponennya adalah sebagai berikut.

- Spark Core adalah bagian inti yang mengatur pembagian tugas, pengelolaan memori, dan pemulihan saat terjadi kegagalan.
- Spark SQL adalah modul untuk mengolah data berbentuk tabel menggunakan perintah SQL.
- Structured Streaming adalah modul untuk menganalisis data yang terus mengalir secara real-time.
- Mllib adalah pustaka untuk machine learning dalam skala besar.
- GraphX adalah modul untuk menganalisis data jaringan atau graf, misalnya hubungan antar pengguna media sosial.

Spark dapat ditulis dengan bahasa Scala, Java, Python (PySpark), R, dan SQL. Spark juga menyediakan REPL (Read-Eval-Print Loop), yaitu layar interaktif untuk mencoba perintah dan melihat hasilnya langsung. Dalam menjalankan tugasnya, Spark dapat memakai berbagai pengelola kluster (cluster manager) seperti YARN, Mesos, Kubernetes, atau Standalone.

B. Perbedaan Spark dengan MapReduce

MapReduce adalah model pemrosesan bawaan Hadoop yang membagi pekerjaan menjadi dua fase. Fase **Map** memecah dan memetakan data, sedangkan fase **Reduce** menggabungkan hasilnya (Dean and Ghemawat, 2004). Setiap kali satu fase selesai, MapReduce menuliskan hasil sementara (intermediate data) ke disk. Cara kerjanya mirip seseorang yang mencatat hasil hitungan di kertas setiap kali selesai satu langkah, lalu membaca kertas itu lagi sebelum melanjutkan. Proses tulis dan baca yang berulang ini memakan banyak waktu. Hambatan tersebut dikenal sebagai I/O bottleneck dan sangat terasa pada algoritma machine learning yang mengulang proses puluhan kali.

Spark mengatasi hambatan tersebut dengan menyimpan data sementara di RAM dan merencanakan seluruh pekerjaan dalam bentuk DAG. Perbandingan keduanya dirangkum pada Tabel 1.

Table 1 Perbandingan Hadoop MapReduce dan Apache Spark

Aspek	Hadoop MapReduce	Apache Spark
Penyimpanan data	Disk fisik (HDFS)	Memori (RAM)
Model eksekusi	Batch dua tahap (Map dan Reduce) yang berjalan terputus-putus	Berjalan kontinu dengan DAG pipelining
Data sementara	Ditulis ke disk setiap akhir fase	Disimpan di memori
Kecepatan	Lebih lambat karena banyak akses disk	Hingga 10 sampai 100 kali lebih cepat
Toleransi kegagalan	Replikasi data yang berat (umumnya 3 salinan)	Lineage graph, yaitu menghitung ulang bagian yang hilang
Kemudahan kode	Rumit dan panjang (verbose Java)	Ringkas dengan API tingkat tinggi
Jenis pemrosesan	Batch	Batch, interaktif, streaming, machine learning, dan graf

Dua hal membuat Spark unggul. Pertama, data sementara tersimpan di RAM sehingga Spark tidak perlu menunggu disk. Kedua, kode Spark jauh lebih singkat. Program penghitung kata yang memerlukan puluhan baris Java pada MapReduce dapat ditulis dalam beberapa baris saja di Spark.

Spark dan Hadoop sebenarnya dapat bekerja bersama. Spark umumnya dijalankan di atas YARN, yaitu sistem pengelola sumber daya Hadoop, dan membaca data dari HDFS (White, 2015). Hadoop berkembang menjadi platform yang dapat menampung banyak mesin pemrosesan sekaligus, seperti MapReduce, Spark, Streaming, dan Graph, pada kluster yang sama.

C. Data yang Dapat Diolah dengan Spark dan MapReduce

Spark dan MapReduce sama-sama sanggup mengolah data berukuran sangat besar yang tersebar di banyak komputer. Data tersebut dapat dikelompokkan menjadi tiga jenis.

- Data terstruktur adalah data yang tersusun rapi dalam baris dan kolom, misalnya tabel basis data, berkas CSV, serta format Parquet dan ORC.

- Data semi-terstruktur adalah data yang memiliki pola tetapi tidak berbentuk tabel, misalnya JSON, XML, dan log aplikasi.
- Data tidak terstruktur adalah data tanpa format baku, misalnya dokumen teks, log mentah, dan citra.

Sumber datanya antara lain HDFS, Amazon S3, HBase, Cassandra, dan basis data yang dihubungkan melalui JDBC. Kecocokan kedua teknologi terhadap jenis pekerjaan dirangkum pada Tabel 2.

Table 2 Kecocokan MapReduce dan Spark terhadap jenis pekerjaan

Jenis pekerjaan	MapReduce	Spark
Pemrosesan batch (analisis log, ETL, pembuatan indeks)	Sesuai	Sesuai
Query interaktif	Kurang sesuai	Sesuai
Algoritma iteratif (machine learning)	Kurang efisien	Sangat sesuai
Data aliran real-time (streaming)	Kurang sesuai	Sesuai
Analisis graf	Kurang efisien	Sesuai

Ringkasnya, MapReduce cocok untuk pekerjaan batch yang membaca data satu kali lalu menuliskan hasilnya. Spark menangani pekerjaan batch tersebut dan juga sanggup mengolah data real-time, menjalankan query interaktif, serta melatih model machine learning.

D. Resilient Distributed Dataset (RDD)

1. Pengertian RDD

RDD adalah singkatan dari Resilient Distributed Dataset. RDD merupakan struktur data dasar di Spark yang berisi kumpulan data yang dibagi menjadi beberapa potongan (partisi) dan disebar ke banyak komputer dalam kluster, sehingga potongan-potongan tersebut dapat diolah secara paralel (Zaharia et al., 2012). Bayangkan sebuah buku tebal yang dibagi per bab, lalu setiap bab dibaca oleh orang yang berbeda pada waktu yang sama. Buku itu adalah RDD, dan setiap bab adalah partisinya. DataFrame dan Dataset yang sering digunakan saat ini pada akhirnya diubah menjadi RDD oleh Spark di balik layar.

2. Tiga Sifat Utama RDD

- Resilient (tangguh), RDD dapat pulih sendiri saat sebuah komputer mengalami kegagalan, tanpa perlu menyalin data ke disk. Pemulihan ini memanfaatkan lineage graph.
- Distributed (terdistribusi), data dipecah menjadi partisi dan disebar ke puluhan sampai ribuan worker node.
- Immutable (tidak dapat diubah), setelah sebuah RDD dibuat, isinya terkunci dan hanya dapat dibaca. Jika data perlu diubah, Spark membuat RDD baru dari RDD lama.

3. Lineage Graph sebagai Penjamin Ketahanan

Cara lama mengamankan data adalah menyalinnya sebanyak tiga kali ke disk, dan cara itu berat serta lambat. Spark memilih cara lain. Spark mencatat seluruh riwayat langkah (transformasi) yang membentuk sebuah RDD, mirip resep masakan yang tertulis lengkap. Jika satu partisi hilang, Spark membuka catatan tersebut dan memasak ulang

partisi itu dari data induknya. Proses ini ringan dan sangat cepat. Sebagai contoh, jika RDD B rusak, Spark menghitung ulang RDD B dari RDD A.

4. Transformations dan Actions

Operasi pada RDD dibagi menjadi dua jenis. **Transformations** menerima sebuah RDD dan menghasilkan RDD baru, misalnya map(), filter(), dan flatMap(). Operasi ini bersifat lazy, yang berarti Spark belum mengolah data dan hanya mencatat rencananya. Ibarat menyusun resep, bahan belum disentuh. **Actions** meminta hasil akhir, misalnya count(), collect(), dan save(). Operasi ini bersifat eager. Saat action dipanggil, Spark langsung menjalankan semua rencana yang telah dicatat. Ibarat memasak, kompor baru dinyalakan setelah resep selesai disusun.

5. Narrow dan Wide Dependencies

Transformations dibedakan lagi berdasarkan arah perpindahan datanya. Pada **narrow dependencies**, setiap partisi input hanya menghasilkan satu partisi output sehingga setiap komputer cukup mengerjakan datanya masing-masing. Prosesnya cepat dan tidak ada data yang berpindah antar komputer. Contohnya map dan filter. Pada **wide dependencies**, satu partisi input menyumbang data ke banyak partisi output, sehingga komputer-komputer harus saling bertukar data melalui jaringan. Proses ini disebut **shuffle**. Shuffle memakan waktu dan sumber daya yang besar. Contohnya groupByKey dan reduceByKey.

E. Directed Acyclic Graph (DAG)

1. Pengertian DAG

DAG adalah peta langkah kerja yang disusun Spark dari kode yang kita tulis. Pengertiannya dapat dipecah per kata. **Directed** berarti setiap langkah memiliki arah dari satu langkah ke langkah berikutnya. **Acyclic** berarti alurnya tidak pernah berputar kembali ke langkah sebelumnya. **Graph** berarti seluruh langkah digambarkan sebagai simpul dan garis penghubung. Setiap simpul adalah sebuah RDD, dan setiap garis adalah transformasi yang menghasilkan RDD tersebut.

2. Lazy Evaluation

Spark menyusun DAG terlebih dahulu dan baru menjalankannya ketika sebuah action dipanggil. Cara kerja ini disebut lazy evaluation. Penundaan tersebut menguntungkan karena tiga alasan. Pertama, Spark dapat melihat seluruh alur kerja dari awal sampai akhir. Kedua, DAG Scheduler dan Catalyst Optimizer dapat menyusun ulang langkah-langkah agar lebih efisien. Ketiga, Spark dapat menggabungkan beberapa filter dan membuang langkah yang tidak diperlukan sebelum data dipindahkan ke memori.

3. Stage, Task, dan Pipelining

Setelah DAG terbentuk, DAG Scheduler memotongnya menjadi beberapa **Stage**. Pemotongan dilakukan pada setiap operasi shuffle. Di dalam satu stage, beberapa operasi narrow dependencies yang berurutan digabung menjadi satu paket kerja melalui **pipelining**. Satu rekaman data langsung melewati rangkaian operasi sekaligus, misalnya map, filter, lalu map, tanpa menyimpan hasil antara setiap langkah. Dengan demikian, Spark menghemat memori dan waktu komputasi. Setiap stage kemudian dibagi lagi menjadi **Task**, yaitu unit kerja terkecil yang dikerjakan pada satu partisi data. Hubungan ketiganya berjenjang sebagai berikut: satu pekerjaan (job) terdiri dari beberapa stage, dan setiap stage terdiri dari banyak task.

F. Alur Kerja Apache Spark

1. Tiga Komponen Utama

Aplikasi Spark berjalan sebagai sekumpulan proses pada kluster yang dikoordinasikan oleh objek SparkSession. Sebagai analogi, anggap sebuah proyek konstruksi yang memiliki manajer, bagian pengadaan, dan para pekerja.

- Driver Process (manajer proyek), proses utama yang berjalan pada komputer tempat fungsi main() dipanggil. Driver menyimpan status aplikasi, mengubah kode pengguna menjadi DAG, lalu membagikan task kepada executor.
- Cluster Manager (bagian pengadaan), pihak yang berhubungan dengan perangkat keras fisik untuk meminta CPU dan memori. Komponen ini dapat berupa YARN, Mesos, Kubernetes, atau Standalone.
- Executor Process (pekerja), proses yang mengerjakan task secara paralel, menyimpan data cache di memorinya sendiri, dan terus melaporkan status serta heartbeat kepada Driver.

2. Empat Tahap Eksekusi

- Inisiasi, pengguna menjalankan perintah spark-submit. Kode aplikasi dievaluasi dan Driver memulai SparkSession.
- Perencanaan, driver menyusun DAG dari operasi logis, lalu memecahnya menjadi Stages dan Tasks.
- Negosiasi, driver meminta sumber daya kepada Cluster Manager agar Executors dapat diluncurkan.
- Eksekusi, tasks dikirim ke Executors dan dikerjakan secara paralel. Hasil akhir dikembalikan ke Driver atau disimpan ke storage.

3. Contoh pada Program Word Count

Contoh berikut adalah program PySpark untuk menghitung kemunculan setiap kata dalam sebuah berkas teks.

```
lines = sc.textFile("hdfs:///data/teks.txt")
words = lines.flatMap(lambda l: l.split(" "))
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs:///output/hasil")
```

Empat baris pertama berisi transformations sehingga Spark baru mencatat rencananya di dalam DAG. Operasi flatMap dan map termasuk narrow dependencies, sehingga keduanya digabung dalam satu stage melalui pipelining. Operasi reduceByKey termasuk wide dependency yang memicu shuffle, sehingga di titik ini terbentuk stage kedua. Eksekusi nyata dimulai pada baris terakhir karena saveAsTextFile adalah sebuah action. Saat itu Driver meminta executor dari Cluster Manager, mengirimkan task, dan menyimpan hasilnya ke HDFS. Apabila sebuah partisi hilang di tengah proses, Spark menghitungnya ulang melalui lineage graph.

G. Kesimpulan

Apache Spark adalah mesin pengolah data terdistribusi yang bekerja di dalam memori sehingga hambatan akses disk pada MapReduce dapat diatasi. Spark unggul dalam

kecepatan, keringkasan kode, dan keragaman pekerjaan yang meliputi batch, streaming, machine learning, serta graf. RDD berperan sebagai struktur data dasar yang tangguh, terdistribusi, dan tidak dapat diubah. DAG berperan sebagai peta eksekusi yang disusun secara lazy dan dioptimalkan melalui pipelining. Alur kerja Spark melibatkan Driver, Cluster Manager, dan Executor yang bekerja sama melalui empat tahap, yaitu inisiasi, perencanaan, negosiasi, dan eksekusi.

Daftar Istilah Penting

Istilah	Arti Sederhana
Kluster	Kumpulan komputer yang bekerja bersama sebagai satu sistem.
Partisi	Potongan kecil dari data besar yang dikerjakan oleh satu komputer.
Shuffle	Proses tukar-menukar data antar komputer melalui jaringan.
Lazy evaluation	Spark menunda pengolahan data sampai ada action yang meminta hasil.
Lineage graph	Catatan langkah pembentuk RDD yang dipakai untuk memulihkan data yang hilang.
Stage dan Task	Stage adalah kelompok langkah kerja, task adalah tugas pada satu partisi.

H. Daftar Pustaka

Apache Software Foundation (n.d.) *Apache Spark documentation*. Available at: https://spark.apache.org/docs/latest/ (Accessed: 1 October 2026).

Chambers, B. and Zaharia, M. (2018) *Spark: the definitive guide: big data processing made simple*. Sebastopol, CA: O'Reilly Media.

Dean, J. and Ghemawat, S. (2004) 'MapReduce: simplified data processing on large clusters', in *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04)*. San Francisco, CA: USENIX Association, pp. 137–150.

Program Studi S1 Teknik Informatika (2026) *Era komputasi memori: pengantar Apache Spark dan RDD*. Slide kuliah Big Data. Surabaya: Universitas Negeri Surabaya.

White, T. (2015) *Hadoop: the definitive guide*. 4th edn. Sebastopol, CA: O'Reilly Media.

Zaharia, M. et al. (2012) 'Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing', in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*. San Jose, CA: USENIX Association, pp. 15–28.